

DecL Cheat Sheet (1/3): agg and pnl

An Aggregate compound distribution has the clause structure:

agg <NAME> <EXPOSURE> <LAYERS*> <SEVERITY> <OCC_RE*> <FREQUENCY> <AGG_RE*> <APPROX*> <TRAILER*>

Key: <INPUT> user input; lower_case a DecL keyword; CLAUSE a sub-clause; options a|b|c; inf infinity; * optional. Build with build('agg ...').

1. Name & references

agg <NAME> ...

sev <NAME> <SEVERITY> *standalone severity*

Names match [a-zA-Z][._:~a-zA-Z0-9-].*

Reference stored objects: agg.<NAME>, sev.<NAME>, dist.<NAME>; clone with agg <NEW> agg.<OLD>.

2. Exposure

<EL> loss

<PREMIUM> premium at <LR> lr

<EXPOSURE> exposure at <RATE> rate

<CLAIMS> claims

dfreq <OUTCOMES> <PROBS*> *fixed empirical freq*

Outcomes [1 2 3 4] or [2:10:2]; probs [.5 .25 .25] or omitted = equally likely.

3. Limit / layers (optional)

<LIMIT> xs <ATTACHMENT>

tower [<BREAKS>]

Occurrence limits on ground-up severity, unlimited reinstatements, losses conditional on attaching by default.

4. Severity (continuous)

sev <DIST> <MEAN> cv <CV>

sev <DIST> <SHAPE1> <SHAPE2*>

sev sev.<NAME> *built-in*

<SCALE> * SEV + <LOC> *scale & shift*

SEV wts [<W>] *mixture*

SEV splice [<LB> <UB>] *conditional in layer*

SEV ! *unconditional (when ATTACH>0)*

ssev ... *spliced severity*

5. Severity (discrete)

dsev <OUTCOMES> <PROBS*> *discrete*

<DIST> xps [<X>] [<P>] *empirical points*

picks [<X>] [<P>] *match layer picks*

dsev [1:6] is a fair die; xps attaches outcomes/probabilities to a parametric severity.

6. Frequency

poisson, bernoulli, fixed, geometric,

logarithmic,

binomial <P>, negbin <VAR_MULT>, neyman

<CLAIMS/OCC>, pascal <CV> <CLAIMS/OCC>

mixed <MIX> <SHAPE1> <SHAPE2*>

MIX = gamma|delaporte|ig|sig|sichel|beta

FREQ zt *zero-truncated*

FREQ zm <PO> *zero-modified, $\Pr(N=0)=p_0$*

7. Occurrence reinsurance

occurrence net of LAYERS

occurrence ceded to LAYERS

LAYER = <LIMIT> xs <ATTACH>

| <SHARE> so <LIMIT> xs <ATTACH>

| <PCT> po <LIMIT> xs <ATTACH>

LAYERS = LAYER and LAYER ... *or tower [<BREAKS>]*

so = share of, po = part of; $0 \leq \text{share} \leq 1$.

8. Aggregate reinsurance

aggregate net of LAYERS

aggregate ceded to LAYERS

aggregate (net of|ceded to) tower [<BREAKS>]

Same LAYER syntax as occurrence, applied to the aggregate loss.

9. Approximate (optional)

approximate exact (*default*)

approximate sgamma *shifted gamma*

approximate slognorm *shifted lognormal*

Alias approx. Replaces freq x sev convolution with a single severity matched to the first three aggregate moments.

10. PnL (premium – loss)

pnl <NAME> <PREMIUM> prem - <AGG-BODY>

Profit-and-loss = premium minus loss. The premium is one deterministic amount subtracted once for the book (aggregate-level shift), unlike a constant in sev (per claim). The body after prem - is an ordinary aggregate; a bare lr binds to the stated premium.

pnl P 1000 prem - 700 loss sev slognorm 50 cv 2 poisson

11. Trailer, vectors, math

Trailer note{free text}, hints{bs=100; log2=17} (*build settings; both optional, order-free*)

Vectorize [1 2 3] claims; [100 200] premium at [.8 .7] lr; [250 250 500] xs [0 250 500] zip; [1 3] * slognorm [.7 1.5] wts [.6 .4]. *Vectors broadcast; layers / exposure zip.*

Math 1/2, 3**4, exp(2), 12.4%. *Warning: minus binds to the number, $-4^2 = 16$ (no unary minus).*

Decl Cheat Sheet (2/3): port, mv, netceded

These declarations build collections and couplings of aggregates. Each component is an ordinary agg (or pnl) declaration.

1. Portfolio (port)

```
port <NAME> <TRAILER*>
  agg <UNIT1> ...
  agg <UNIT2> ...
  ...
```

A *Portfolio* is a list of independent units sharing a discretization grid. Units may be *agg* or *pnl* lines, or references *agg*. <NAME>.

2. port example

```
port MyBook
  agg A 100 claims sev lognorm 50 cv 2
  poisson
  agg B 200 claims sev gamma 30 cv 1 poisson
```

Units are added independently; the portfolio total is their convolution. Per-unit reinsurance is set on each *agg* line.

3. Multivariate (mv)

```
mv <NAME> <SHARED-EXPOSURE>
  agg <A> ...
  agg <B> ...
  copula <KIND> <P*>
  <FREQUENCY*>
```

Alias *multivariate*. Couples exactly two component severities by a copula and accumulates them with a shared outer frequency via a 2D FFT. Frequency optional (default poisson).

4. Copula clause

```
copula <KIND> <P> one-parameter
copula <KIND> parameter-free
omitted = copula independent

KIND = normal|gumbel|clayton|fgm|independent

mv M 100 claims agg A ... agg B ... copula
gumbel 0.4
```

5. netceded

```
netceded <AGG>
```

Build the bivariate (*net*, *ceded*) joint outcome of one reinsured aggregate — the two coupled marginals are the net and ceded positions of the same loss, so their dependence is exact (comonotone within the layer). Lets you see net and ceded jointly, not just their marginals.

```
netceded agg G 100 claims sev lognorm 50 cv
2 occurrence net of 500 xs 500 poisson
```

DecL Cheat Sheet (3/3): distortion

A distortion g defines a coherent risk measure / pricing functional. Declare one by kind and its natural parameters; the kind $\rightarrow$ parameter mapping lives on each Distortion subclass.

1. Declaration

```
distortion <NAME> <KIND> <PARAMS>
```

Alias *dist.* <PARAMS> is a flat list of the kind's natural parameters (see table). Reference a stored distortion as *dist.* <NAME>.

```
distortion D1 ph 0.9
dist D2 wang 0.25
dist D3 power 0 100 2
```

2. Kinds & parameters

```
ph <a> proportional hazard
wang <lam> Wang / normal shift
dual <b> dual moment
tvar <p> tail VaR at p
ccoc <r> constant cost of capital
bitvar <p0> <p1> <w1> two-point
beta <a> <b>
power <x0> <x1> <alpha>
```

3. More kinds

```
wtdtvar ... weighted TVaR (knots)
c1l <b> capped log-linear
clin <slope> capped linear
lep <r> leverage-equivalent pricing
ly <r> linear yield
```

List available kinds with
`Distortion.available_distortions()`.

4. Combinators

```
distortion <NAME> minimum dist.<A> dist.<B>
...
distortion <NAME> mixture dist.<A> dist.<B>
... wts [<W>]
```

minimum = pointwise min of the referenced distortions;
mixture = weighted average (weights via *wts*).

5. Usage

Apply a distortion to price an aggregate or portfolio:

```
a = build('agg A 100 claims sev lognorm 50
cv 2 poisson')
d = build('distortion D ph 0.9')
a.price(0.99, d)
```

or reference it inside pricing calls by *dist.* <NAME>. A distortion satisfies $g(0) = 0$, $g(1) = 1$, increasing and concave.

Underwriter Class Cheat Sheet

 `Underwriter(*, name='Rory', databases=None, update=cfg, log2=cfg, debug=False)`^[1]

The Underwriter manages the creation of Aggregate, Severity, Portfolio, and Distortion objects, maintains a knowledge base of named DecL declarations keyed by (kind, name), and provides the interface to the Lark-based DecL parser. The convenience method  `build`^[2] is the primary entry point; `uw(program)` delegates to it. The following tables show all

 methods, and fields or properties (used interchangeably). Comments elucidate the meaning of more obscure entries.

1. Specification & creation

name identifying name for this Underwriter instance
databases the load request: *name / path / glob*, or 'default', 'user', 'all'; a bare `Underwriter()` loads nothing
update if true, constructed objects are updated (probabilities computed); defaults to `config`
log2 default $\log_2$ of the number of buckets used to discretize; defaults to `config`
debug if true, print parser debug messages

2. Update

 `build`^[2] (primary entry point: parse a single-output DecL program and return the constructed object),
 `build_many` (parse a multi-output program, construct each, smart-update; returns `list[ParsedProgram]`),
knowledge (the knowledge base as a (kind, name)-indexed DataFrame, lazy-loaded),  `load` (resolve a request and read matching .agg files into the knowledge),  `add_entry` (add or overwrite one parsed declaration in the knowledge),  `to_agg` (export selected knowledge entries to a .agg file; re-loads cleanly),  `reload` (reset to as-created state and re-read the configured request), *lexer*, *parser* (the DecL lexer / parser objects)

3. Moments

None

4. Statistical functions

None

5. Validation

 `interpret_file` (parse every DecL program in a .agg / .csv file and return per-line error info; collects *all* errors rather than aborting — validate a file before installing it), `test_suite_file` (path to the bundled test suite .agg, or None)^[3]

6. Output dataframes

None

7. Reinsurance

None

8. Visualization

None

9. Risk and pricing

None

10. Approximations

None

11. Meta

 `discover` (search the knowledge base by regex and optional kind; optionally describe, plot, or return the objects — the directory view, replacing `qlist/qshow/show/dir`),  `help` (help on methods and properties matching regex),
 `available_databases` (list .agg files on the search path: `cwd / user / default`),  `resolve_databases` (dry run: preview the files a request *would* load),
 `show_settings` (print every resolved setting and where its value came from),  `reload_settings` (re-read the config file / environment),
 `write_default_config` (write the annotated config template to `user_dir`), `default_dir` (bundled .agg databases), `user_dir` (~/.aggregate), `version` (aggregate version string)

Notes:

[1]: Arguments are **keyword-only** (leading *): use `databases='test_suite'`, not `Underwriter('test_suite')`. `update` and `log2` default to the [build] section of `config.toml`.

[2]: Also exported as `from aggregate import build`. The module-level `build` pre-loads `build.databases` (`test_suite`) whereas a bare `Underwriter()` starts empty. `build` returns one object, `build_many` a list.

[3]: `aggregate/agg/test_suite.agg` (categories A–O) is now exercised by `uv run pytest`, not by an Underwriter method.

`kind` ∈ {sev, agg, port, distortion}; database search path `cwd` → `user_dir` → `default_dir`.



Severity Class Cheat Sheet

 Severity(*sev_name*, *exp_attachment*=None, *exp_limit*=inf, *sev_mean*=0, *sev_cv*=0, *sev_a*=nan, *sev_b*=0, *sev_loc*=0, *sev_scale*=0, *sev_xs*=None, *sev_ps*=None, *sev_wt*=1, *sev_lb*=0, *sev_ub*=inf, *sev_conditional*=True, *name*=")

A Severity wraps a frozen `scipy.stats` rv (in `fz`) for a single ground-up loss, optionally cut to a layer by the attachment / limit clause. Mixtures and the `wt` weight are handled by Aggregate. The following tables show all  methods, and fields or properties (used interchangeably). Comments elucidate the meaning of more obscure entries.

1. Specification & creation

name *object name*
sev_name *scipy dist name, or (c|d)histogram*
sev_kind, *long_name* *resolved labels*
sev_mean, *sev_cv* *ground-up mean / CV, or*
sev_a, *sev_b* *shape parameters 1, 2*
sev_loc, *sev_scale* *shift and scale*
sev_xs, *sev_ps* (*d*)*histogram outcomes / probs*
sev_wt *mixture weight (used by Aggregate)*
sev_conditional *losses conditional on attaching the layer (default) or unconditional*
sev_signed, *sev_reflect* *allow negative / reflect about 0*
note, *hints*, *program provenance*

2. Update

exp_attachment, *exp_limit* *layer inputs*
attachment, *limit*, *detachment* *resolved layer*
sev_lb, *sev_ub* *conditional support bounds*
pattach, *pdetach* *Pr(attach), Pr(detach)*

3. Moments

sev1, *sev2*, *sev3* (first three raw moments),  *moms* (*m*, *cv*, *skew*),  *mean*,  *median*,  *std*,  *var*,  *stats*,  *moment*,  *generic_moment*,  *moment_type*,  *support*

4. Statistical functions

 *cdf*,  *sf*,  *pdf*,  *logcdf*,  *logsf*,  *logpdf*,  *ppf*,  *isf*,  *rvs*,  *interval*,  *expect*,  *entropy*,  *vecentropy*,  *nnlf*

5. Validation

None

6. Output dataframes

None

7. Reinsurance

None (layer cut only; see Update)

8. Visualization

 *plot*

9. Risk and pricing

None

10. Approximations

 *fit*,  *fit_loc_scale*,  *freeze*

11. Meta

fz (the frozen `scipy` rv), *shapes*, *numargs* (`scipy` shape info), *conditional*, *signed*, *bounded*, *tail_class*, *tail_description*, *support_description*, *random_state*, *xtol*

Notes:

A Severity models one ground-up severity curve; frequency, mixing, and reinsurance live on Aggregate.

sev_mean+*sev_cv* or *sev_a*(+*sev_b*) parameterise the shape; *sev_loc*/*sev_scale* then shift/scale.

Abbreviations: *m*=mean, *cv*=coefficient of variation, *sd*=standard deviation, *var*=variance, *skew*(ness).

Aggregate Class Cheat Sheet

m `Aggregate(name, exp_el=0, exp_premium=0, exp_lr=0, exp_en=0, exp_attachment=None, exp_limit=inf, sev_name="", sev_a, sev_b, sev_mean, sev_cv, sev_loc, sev_scale, sev_xs, sev_ps, sev_wt, sev_lb, sev_ub, sev_conditional, occ_reins, occ_kind, freq_name, freq_a, freq_b, freq_zm, freq_p0, agg_reins, agg_kind, approximate, note, hints)`^[0]

The constructor follows the DecL clauses (exposure, severity, occurrence reinsurance, frequency, aggregate reinsurance). Usually built via `build('agg ...')`. The following tables show all

m methods, and fields or properties (used interchangeably). Comments elucidate the meaning of more obscure entries.

1. Specification & creation

`name`, `limit`, `attachment`, `frequency`, `sevs` (list of Severity), `en`, `n` (frequency), `value_type` (loss / premium for pnl), `sev_pick_attachments`, `sev_pick_losses`, `note`, `hints`, `program`, `pprogram` (pretty), `pprogram_html`, `spec`, `spec_ex`

2. Update

`log2`, `bs` (bucket size), `sev_calc` (round / forward / backward), `discretization_calc` (distribution / survival / both), `normalize`, `padding`, `approximate` (exact / sgamma / slognorm), `fzapprox`, `dsev_bucket`, `reins_bucket`, `x_min`, `x_max`, `i0`, **m** `update`, **m** `update_work`, **m** `discretize`, **m** `recommend_bucket`, **m** `picks`, **m** `rescale` (homog. sev / inhomog. freq), **m** `snap`, **m** `unwrap`

3. Moments

est prefix = estimated from FFT
`agg_m`, `agg_cv`, `agg_sd`, `agg_var`, `agg_skew`, `est_m`, `est_cv`, `est_sd`, `est_var`, `est_skew`, `sev_m`, `sev_cv`, `sev_sd`, `sev_var`, `sev_skew`, `est_sev_m`, `est_sev_cv`, `est_sev_sd`, `est_sev_var`, `est_sev_skew`, **m** `freq_pmf`

4. Statistical functions

m `cdf`, **m** `sf`, **m** `pdf`, **m** `pmf`, **m** `q` (lower quantile = VaR), **m** `ppf`, **m** `tvar`, **m** `sev` (exact severity cdf/sf/pdf), **m** `q_sev`, **m** `tvar_sev`, **m** `var_dict`^[1], **m** `sample`

5. Validation

`valid`, `validation_eps`, `describe` (validation stats), **m** `explain_validation`, **m** `aggregate_error_analysis` (agg error over bs), **m** `severity_error_analysis` (trunc. / discr. by component), `bs_window_df`, `bs_description`, `bs_explanation`, `tail_class`, `tail_df`, `tail_description`, `tail_explanation`

6. Output dataframes

`density_df`^[1] (main output), `stats_df`, `info`, `density`, `agg_density`, `sev_density`, `sev_density_df`, `ftagg_density` (FT), `xs`, `xs_sev`, `figure`, bounded *see also Reinsurance*

7. Reinsurance

`occ_kind`, `occ_reins`, `agg_kind`, `agg_reins`, `agg_density_gross/ceded/net`, `sev_density_gross/ceded/net`, `occ_ceder`, `occ_netter`, `agg_ceder`, `agg_netter`, **m** `apply_occ_reins`, **m** `apply_agg_reins`, **m** `reins_description`, `reins_kinds`, `reins_density_df`, `reins_describe`, `reins_stats_df`

8. Visualization

m `plot`, **m** `reins_occ_plot`, **m** `occ_bivariate`, `figure`

9. Risk and pricing

m `price`, **m** `price_pentagon`, **m** `apply_distortion`, **m** `cramer_lundberg` (aka pollaczec_khinchine; prob. of eventual ruin)

10. Approximations

`approximate` (exact / sgamma / slognorm), **m** `entropy_fit`, `fzapprox`

11. Meta

m `help`, **m** `json` (persist), `value_type`, `spec`, `spec_ex`

Notes:

[0]: Full signature also has `sev_signed/reflect`, `agg_premium/shift/reflect` (the pnl affine wrapper), `reins_bucket`, `dsev_bucket`; see help.

[1]: matches Portfolio.

Abbreviations: gc_n = gross (subject), ced_{ed}, net; m=mean, cv=coefficient of variation, sd=standard deviation, var=variance, skew(ness); VaR=value-at-risk; est=FFT estimate.

Portfolio Class Cheat Sheet

 Portfolio(name, spec_list, uw=None)
A Portfolio is a collection of Aggregate units sharing a discretization grid; it adds diversification, capital allocation, and distortion pricing. spec_list is a list of agg specs / programs; usually built via build('port ...'). The following tables show all  methods, and fields or properties (used interchangeably). Comments elucidate the meaning of more obscure entries.

1. Specification & creation

name, agg_list (the units), n_units, unit_names, unit_names_ex, line_names, line_names_ex, line_name_pipe, line_renamer, <unit> (each unit by name, e.g. A), value_type, program, pprogram, pprogram_html, spec, spec_ex,  nice_program,  as_severity,  create_from_sample

2. Update

log2, bs, sev_calc, discretization_calc, normalize, padding, last_update, hash_rep_at_last_update,  update,  recommend_bucket,  best_bucket,  best_window,  snap,  remove_fuzz,  trim_df,  swap_density_df

3. Moments

est prefix = estimated from FFT
agg_m, agg_cv, agg_sd, agg_var, agg_skew, est_m, est_cv, est_sd, est_var, est_skew, ex ($E[X]$)

4. Statistical functions

 cdf,  sf,  pdf,  pmf,  q (VaR),  tvar,  tvar_threshold,  var,  var_dict^[1],  percentiles,  sample, sample_df,  sample_compare,  sample_density_compare

5. Validation

valid, validation_eps, describe,  explain_validation, bs_window_df, bs_description, bs_explanation, tail_class, tail_df, tail_description, tail_explanation

6. Output dataframes

density_df^[1], density, stats_df, info, independent_density_df, independent_stats_df,  unit_density,  unit_density_df,  aligned_unit_density_df,  augmented_df, augmented_dfs, figure, bounded

7. Reinsurance

reins_density_df, reins_describe, reins_stats_df (*per-unit reinsurance is set on each Aggregate*)

8. Visualization

 plot,  scatter, figure

9. Risk and pricing

 price,  price_ccoc,  price_stand_alone,  price_pentagon,  pricing_at,  pentagon_at,  pricing_bounds,  apply_distortion, distortion, distortions, distortion_df, calibration_df,  calibrate_distortion,  calibrate_distortions,  analyze_distortion,  analyze_distortions, allocation_method,  allocation_bounds,  allocation_diagnostics,  bodoff,  add_exa,  add_exa_sample,  make_comonotonic_allocations

10. Approximations

 approximate (fit a single Aggregate to the total)

11. Meta

 help,  json,  save

Notes:
[1]: matches Aggregate.
Abbreviations: m=mean, cv=coefficient of variation, sd=standard deviation, var=variance, skew(ness); VaR=value-at-risk; est=FFT estimate; exa = $E[X_i | X]$ (conditional expected allocation).

Distortion Class Cheat Sheet

m `Distortion(name, shape=None, *, display_name="", **natural)`
Distortion is a factory base class: the call `Distortion('ph', 0.9)` dispatches on name to a registered subclass (`PHDistortion`, `TVaRDistortion`, ...) and returns an instance. A distortion is an increasing, concave $g : [0, 1] \rightarrow [0, 1]$ with $g(0) = 0$, $g(1) = 1$, used to define a coherent risk measure / pricing functional. Create one by family with a named constructor (**s** `Distortion.ph(0.9)`), or via DecL `distortion <NAME> <KIND> <SHAPE>`. The following tables show all **m** methods, **s** static / class methods, and fields or properties (used interchangeably). Comments elucidate the meaning of more obscure entries.

1. Specification & creation

name *the kind string, e.g. 'ph'*
shape *the (single) shape parameter*
kind *registry key for the subclass*
display_name *optional label override*

Named constructors, by family (each **s** static): **s** `ph` (proportional hazard), **s** `wang` (Wang / normal shift), **s** `dual` (dual moment), **s** `power` (power on $[x_0, x_1]$), **s** `beta`, **s** `tvar` (TVaR at level p), **s** `bitvar` ($p_0 < p_1$, weight w on p_1), **s** `ccoc` (constant cost of capital from discount d), **s** `mean` (= TVaR at $p=0$), **s** `max` (= TVaR at $p=1$). *Combine / generate:*
s `mixture` (weighted mixture), **s** `minimum` (pointwise min), **s** `random_distortion`, **s** `random_distortion_ex` (random wtdTVaR), **s** `distortions_from_params` (dict from a calibration table), **s** `available_distortions` (list registered kinds), **s** `decl_spec` (spec dict from a DecL number list)

2. Update

None

3. Moments

None

4. Statistical functions

m `g` (the distortion $g(x)$), **m** `g_inv` (inverse g^{-1}), **m** `g_dual` (dual $1 - g(1 - x)$), **m** `g_prime` (derivative g' , central-difference default), **m** `effective_g` (resolve ask / bid view to the effective g), **m** `make_q` (vector of risk-adjusted probabilities q for pricing), **s** `tvar_terms` (TVaR knot function $\min(s/(1 - p), 1)$)

5. Validation

s `pricing_ok` (the distortion yields well-defined prices), **s** `strict_pricing` (enforce strict concavity / validity), **s** `error` (numerical g vs. closed-form residual; see `describe`)

6. Output dataframes

s `info` (text summary)^[1], **s** `describe`^[1] (moments of g , g^{-1} vs. closed form), **s** `stats_df`^[1], **s** `density_df`^[1], **m** `tvar_info_df` (affine pieces of a weighted-TVaR), **s** `decl_params`, **s** `renamer`

7. Reinsurance

None

8. Visualization

m `plot` (g , dual, and diagonal)^[1], **m** `plot_affine` (upper affine envelope of a wtdTVaR)

9. Risk and pricing

m `price` (distorted ask / bid prices of a distribution), **m** `calibrate` (solve shape so the distorted price hits a premium_target), **s** `premium_target`, **s** `assets` (capital / a), **s** `mass`, **s** `has_mass` (point mass $g(0^+) > 0$), **s** `gini_p` (Gini coefficient)

10. Approximations

None

11. Meta

m `id` (short structural ID string), **s** `med_name`, **s** `long_name`, **s** `param_name` (display labels), **s** `documented` (listed by available_distortions), **m** `min_index`, **m** `quick_gS`, **m** `quick_ra` (internal numerical helpers)

Notes:

[1]: `info`, `describe`, `stats_df`, `density_df`, and `plot` mirror the common reporting interface shared with `Aggregate` and `Portfolio` — the same names do the analogous job on each class.
A distortion satisfies $g(0) = 0$, $g(1) = 1$, increasing and concave; the dual is $g^*(x) = 1 - g(1 - x)$. Distorted price = $\int_0^\infty g(S(x)) dx$.

